[H-01] Buffer Inconsistency Creates Permanent "Zombie" Accounts That Lock Protocol Liquidity

Summary

A critical design flaw exists where the protocol uses different collateralization buffers for position management (`BP_DECREASE_BUFFER = 1.3333x`) versus liquidation (`NO_BUFFER = 1.0x`). This 33.33% gap creates a "zombie zone" where accounts become trapped: they cannot close their positions (failing the stricter buffer check) yet cannot be liquidated (passing the looser check). Combined with force exercise limitations, this can result in **permanent fund lockup**.

Vulnerability Detail

The Panoptic protocol enforces solvency checks at two critical junctures with inconsistent thresholds:

The Two Buffers

```
// PanopticPool.sol:140-143
uint256 internal constant BP_DECREASE_BUFFER = 13_333;  // 1.3333x -
uint256 internal constant NO_BUFFER = 10_000;           // 1.0x - Us
```

When Each Buffer Applies

OPERATION	BUFFER USED	MULTIPLIER
`mintOptions()`	`BP_DECREASE_BUFFER`	1.3333x
`burnOptions()`	`BP_DECREASE_BUFFER`	1.3333x
`liquidate()`	`NO_BUFFER`	1.0x

The Zombie Zone

When an account's collateral ratio falls between 1.0x and 1.3333x:

```
Collateral Ratio:  0.5x    1.0x         1.3333x         2.0x
                    |________|_____________|_______________|
                    INSOLVENT   ZOMBIE ZONE      HEALTHY

                    Can be      Cannot act     Can act
                    liquidated  AND cannot     freely
                                be liquidated
```

Code Flow Analysis

Attempting to close position (burnOptions):

```
// PanopticPool.sol:616
function _validateSolvency(...) {
    // Uses BP_DECREASE_BUFFER (1.3333x)
    if (collateralRatio < BP_DECREASE_BUFFER) revert Errors.AccountI
}
```

Attempting to liquidate:

```
// PanopticPool.sol:962
function liquidate(...) {
    // Uses NO_BUFFER (1.0x)
    if (collateralRatio ≥ NO_BUFFER) revert Errors.NotMarginCalled(
}
```

Impact

Quantified Damage Potential

SCENARIO	LOCKED VALUE	DURATION
Single zombie account	Position notional + collateral	Indefinite
Market stress event	Multiple accounts × avg position	Indefinite
Protocol-wide	Potentially all undercollateralized positions	Indefinite

Cascading Effects

1. **Uniswap Liquidity Lock**: Zombie positions keep liquidity deployed in Uniswap V3, unable to be withdrawn
2. **Premium Accumulation**: Ongoing premium obligations continue accruing with no resolution mechanism
3. **User Fund Freeze**: Collateral remains locked in CollateralTracker with no exit path
4. **Protocol Degradation**: As more accounts enter zombie state, available liquidity decreases

Real-World Example

```
Initial State:
- User deposits $15,000 collateral
- Opens $100,000 notional short put position
- Collateral ratio: 1.5x (healthy)

After Adverse Price Move (-$4,000 loss):
- Effective collateral: $11,000
- Required margin: $10,000
- Collateral ratio: 1.1x

Result:
- Cannot close: 1.1x < 1.3333x (fails BP_DECREASE_BUFFER)
- Cannot liquidate: 1.1x > 1.0x (passes NO_BUFFER)
- $100,000 Uniswap liquidity LOCKED INDEFINITELY
```

Proof of Concept

Mathematical Proof (Unit Test)

```
function test_ZombieZoneBoundaries() public {
    uint256[] memory testRatios = new uint256[](5);
    testRatios[0] = 9999;   // Below 1.0x - LIQUIDATABLE
    testRatios[1] = 10000;  // Exactly 1.0x - ZOMBIE STARTS
    testRatios[2] = 11500;  // Mid zombie - ZOMBIE
    testRatios[3] = 13332;  // Just below 1.3333x - STILL ZOMBIE
    testRatios[4] = 13333;  // Exactly 1.3333x - HEALTHY

    for (uint256 i = 0; i < testRatios.length; i++) {
        bool canAct = testRatios[i] ⩾ BP_DECREASE_BUFFER;
        bool canBeLiquidated = testRatios[i] < NO_BUFFER;
        bool isZombie = !canAct && !canBeLiquidated;
        // Ratios 10000-13332 are ALL zombie
    }
}
```

Test Output

```
=== Zombie Zone Boundaries ===
Ratio 9999  => LIQUIDATABLE
Ratio 10000 => ZOMBIE
Ratio 11500 => ZOMBIE
Ratio 13332 => ZOMBIE
Ratio 13333 => HEALTHY

!!! ZOMBIE STATE CONFIRMED !!!
Account with ratio 1.1111x:
  - Cannot close position (fails 1.3333x check)
  - Cannot be liquidated (passes 1.0x check)
```

Run the PoC

```
FOUNDRY_PROFILE=poc forge test --match-contract H01_ZombieZone_UnitT
```

Result: 5 tests passed

Tools Used

- Manual code review
- Foundry for PoC development
- Mathematical analysis of buffer constants

Recommended Mitigation

Option 1: Unify Buffers (Recommended)

```
// Use single buffer for both operations
uint256 internal constant SOLVENCY_BUFFER = 12_000; // 1.2x

// Apply consistently in both _validateSolvency and liquidate
```

Option 2: Implement Grace Period

```
// Allow users in zombie zone a grace period to add collateral
mapping(address ⇒ uint256) public zombieDeadline;

function enterZombieZone(address account) internal {
    if (zombieDeadline[account] == 0) {
        zombieDeadline[account] = block.timestamp + 24 hours;
    }
}

// After grace period, allow liquidation at BP_DECREASE_BUFFER
```

Option 3: Reduce Gap

```
// Reduce the gap to minimize zombie zone
uint256 internal constant BP_DECREASE_BUFFER = 11_000;  // 1.1x (was
uint256 internal constant NO_BUFFER = 10_000;           // 1.0x

// Only 10% gap instead of 33%
```

Critical Note

Any mitigation must ensure backward compatibility and consider positions opened under the old regime.